Music Beat Detection using Digital Signal Processing

Prepared by

Vansh Champaneri
Roll No: 23BEC027

Dev Dobariya
Roll No: 23BEC038



Department of Electronics and Communication Engineering,
Institute of Technology
Nirma University
Ahmedabad 382481

March 24, 2025

Contents

1 Abstract **2**

2 Introduction **2**

3 Problem Description **2**

4 Algorithm for Music Beat Detection **2**

5 Flowchart **4**

6 Methodology **5**
6.1 Reading the Audio File 5
6.2 Preprocessing and Downsampling 5
6.3 Bandpass Filtering 5
6.4 FFT and Energy Computation 5
6.5 Peak Detection and BPM Estimation 6

7 DSP Concepts Used **6**
7.1 Fast Fourier Transform (FFT) 6
7.2 Short-Time Energy (STE) 6
7.3 Spectral Flux (SF) 6
7.4 Beats Per Minute (BPM) Calculation 6

8 Applications **7**
8.1 Music Analysis and Remixing 7
8.2 Dance and Fitness Applications 7
8.3 Music Visualization 7
8.4 Automatic Music Tagging and Categorization 7
8.5 Speech and Audio Processing 7

9 Observations **7**
9.1 Performance on Simple vs. Complex Audio 7
9.2 Challenges with Voice and Instrumental Complexity 8
9.3 BPM Estimation Variability 8

10 Challenges and Possible Improvements **8**
10.1 Challenges Faced 8
10.1.1 Noise Interference 8
10.1.2 Variability in Music Tempo 8
10.1.3 Threshold Selection for Beat Detection 8
10.2 Possible Improvements 8
10.2.1 Adaptive Thresholding 8
10.2.2 Machine Learning Integration 8
10.2.3 Filter Design for Noise Reduction 9

11 Conclusion **9**

12 MATLAB Code for Beat Detection **9**

1 Abstract

Beat detection, which has historically been common in rhythm-oriented music analysis, synchronization, and effect, is a crucial task in digital signal processing. The project described is specifically concerned with using MATLAB to detect beats in an audio signal. The method used is pre-processing audio by filtering important frequency components and using the Fast Fourier Transform (FFT) to analyze changes in energy patterns. Energy peaks are found in order to estimate beats per minute (BPM). With accurate tempo estimation, the method was also confirmed with a metronome sound file. To enhance detection in multiplex music files, adaptive filtering and more advanced machine learning algorithms may be introduced in the future.

2 Introduction

One of the important parts of digital signal processing, beat detection has many applications across diverse domains such as audio synchronizing, rhythm-related apps, and music analysis. Detecting beats in an audio signal enables one to predict the rate of a musical work. The process of detecting beats, and most importantly, correct detection of beats, is greatly helpful in the domains of automatic music transcription, audio visualization, and DJing-related applications. In this instance, an algorithm based on MATLAB is used to identify beats from an audio signal and calculate the beats per minute (BPM). The steps include reading an audio file, filtering frequency components of the appropriate order, analyzing energy variations with the use of the Fast Fourier Transform (FFT), and identifying peaks in energy to specify beats. The algorithm is validated with a metronome audio file, which provides a controlled environment for its accuracy testing.

The process of beat detection such as preprocessing techniques, energy computation, and peak detection method is clearly explained in this report. The effectiveness of the method is shown from the results, and enhancements that are described detail how to enhance robustness under challenging audio conditions.

3 Problem Description

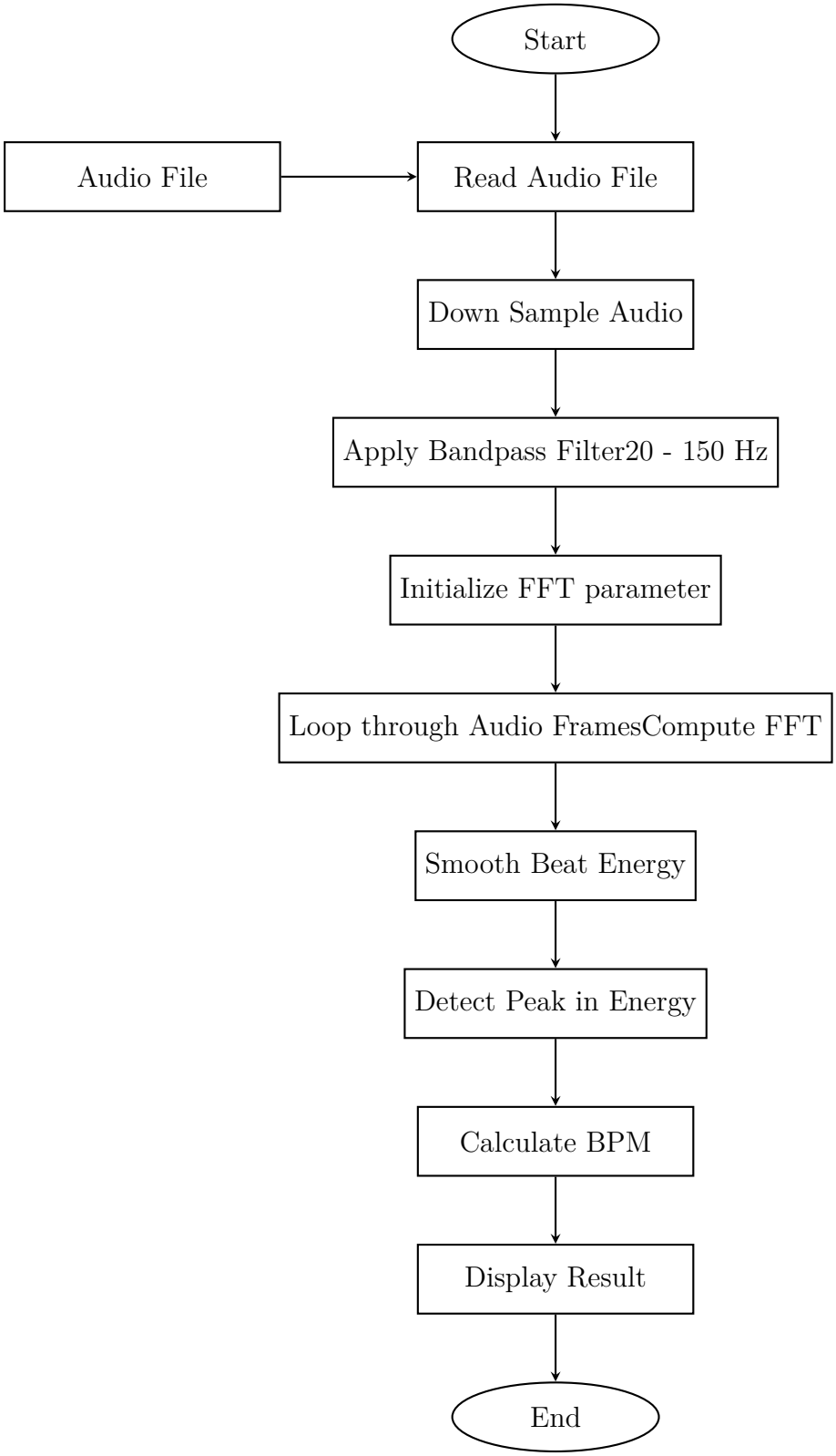
Beat detection is crucial to music analysis, audio processing applications, and synchronization. The problem is accurately detecting beat positions in a signal regardless of instrument dynamics, tempo, and ambient noise changes. The goal of this project is to implement a MATLAB-based beat detection algorithm that takes an input audio file, detects frequencies corresponding to beats, and estimates tempo (BPM) using digital signal processing. Beat detection accuracy relies on proper filtering, energy analysis, and peak detection techniques.

4 Algorithm for Music Beat Detection

1. **Read the Audio File**
    - Load the input audio file using `audioread()`.
    - Convert stereo audio to mono by averaging channels.

2. **Preprocessing and Downsampling**
    - Define a target sampling frequency (2000 Hz).
    - If the original sampling frequency is higher, downsample the audio.
3. **Apply Bandpass Filtering**
    - Design a bandpass filter with a range of 20 Hz to 150 Hz.
    - Apply the filter to the audio signal using `filtfilt()`.
4. **Divide the Signal into Frames and Compute FFT**
    - Set frame size (1024 samples) and hop size (512 samples).
    - Loop through the signal in overlapping frames:
        - Apply a Hamming window to the frame.
        - Compute the FFT and extract magnitude values.
        - Calculate and store the total energy of each frame.
5. **Smooth the Energy Signal**
    - Apply a moving average filter to smooth the computed energy.
6. **Peak Detection for Beat Identification**
    - Set a threshold as a percentage of the maximum energy.
    - Detect peaks in the energy signal using `findpeaks()`.
    - Store the positions of detected beats.
7. **BPM Estimation**
    - Count the number of detected beats.
    - Compute BPM using the total duration of the audio.
8. **Results and Visualization**
    - Display the detected number of beats and estimated BPM.
    - Plot the beat energy signal along with detected peaks.

5 Flowchart


Start
Audio File
Read Audio File
Down Sample Audio
Apply Bandpass Filter20 - 150 Hz
Initialize FFT parameter
Loop through Audio FramesCompute FFT
Smooth Beat Energy
Detect Peak in Energy
Calculate BPM
Display Result
End

6 Methodology

6.1 Reading the Audio File

The MATLAB function `audioread` is used to load the input audio file. To simplify processing, stereo files are converted to mono:

```
[audio, fs] = audioread('Kick␣Drum␣Metronomes␣-␣60␣BPM.mp3');
audio = mean(audio, 2); % Convert stereo to mono
```

6.2 Preprocessing and Downsampling

To reduce computational complexity, the audio is downsampled to a target frequency of 2000 Hz:

```
targetFs = 2000;
if fs > targetFs
    audio = resample(audio, targetFs, fs);
    fs = targetFs;
end
```

6.3 Bandpass Filtering

A bandpass filter (20–150 Hz) is applied to isolate beat-related frequencies:

```
bpFilt = designfilt('bandpassiir', 'FilterOrder', 6, ...
    'HalfPowerFrequency1', 20, 'HalfPowerFrequency2', 150, 'SampleRate', fs
        );
filteredAudio = filtfilt(bpFilt, audio);
```

6.4 FFT and Energy Computation

The signal is divided into overlapping frames, and FFT is applied to compute beat energy:

```
frameSize = 1024;
hopSize = 512;
numFrames = floor((length(filteredAudio) - frameSize) / hopSize) + 1;
beatEnergy = zeros(numFrames, 1);

for frame = 1:numFrames
    startIdx = (frame - 1) * hopSize + 1;
    endIdx = startIdx + frameSize - 1;
    if endIdx > length(filteredAudio)
        break;
    end
    signalFrame = filteredAudio(startIdx:endIdx) .* hamming(frameSize);
    fftResult = fft(signalFrame);
    fftMagnitude = abs(fftResult(1:frameSize/2));
    beatEnergy(frame) = sum(fftMagnitude.^2);
end
```

6.5 Peak Detection and BPM Estimation

The energy signal is smoothed, and peak detection is performed:

```
smoothedBeatEnergy = movmean(beatEnergy, 5);
threshold = 0.3 * max(smoothedBeatEnergy);
[~, beatLocs] = findpeaks(smoothedBeatEnergy, 'MinPeakHeight', threshold,
    ...
    'MinPeakDistance', round(fs * 60 / 120 / hopSize));

beatCount = length(beatLocs);
bpm = (beatCount / (length(filteredAudio) / fs)) * 60;
```

7 DSP Concepts Used

This project utilizes key Digital Signal Processing (DSP) concepts for music beat detection:

7.1 Fast Fourier Transform (FFT)

- Converts the time-domain audio signal into the frequency domain.
- Helps analyze energy distribution across frequencies to identify rhythmic patterns.

7.2 Short-Time Energy (STE)

- Measures the energy of short frames of the signal.
- Used to detect sudden energy spikes corresponding to beats.

7.3 Spectral Flux (SF)

- Captures changes in spectral energy over time.
- Helps in identifying beats by detecting significant variations in frequency components.

7.4 Beats Per Minute (BPM) Calculation

BPM is estimated based on detected beats:

$$BPM = \left(\frac{\text{Number of Detected Beats}}{\text{Total Duration (seconds)}} \right) \times 60 \tag{1}$$

The Total Duration is calculated as:

$$\text{Total Duration} = \frac{\text{Total Samples}}{\text{Sampling Frequency}(F_s)} \tag{2}$$

Number of Detected Beats is derived from peak detection in energy or spectral flux signals.

8 Applications

Music beat detection has a wide range of applications in various fields:

8.1 Music Analysis and Remixing

- Helps DJs and music producers synchronize beats for remixing.
- Enables automatic beat-matching in DJ software.

8.2 Dance and Fitness Applications

- Used in dance choreography software to sync movements with music.
- Fitness apps use beat detection to match workouts with high-energy songs.

8.3 Music Visualization

- Generates real-time visual effects that sync with beats in music players.
- Used in concerts and media productions to enhance audience engagement.

8.4 Automatic Music Tagging and Categorization

- Helps classify music by genre based on tempo and rhythm patterns.
- Used in music streaming platforms for personalized recommendations.

8.5 Speech and Audio Processing

- Assists in separating speech from background music in audio processing.
- Useful in speech enhancement techniques and audio restoration.

9 Observations

After implementing and testing the beat detection algorithm, the following observations were made:

9.1 Performance on Simple vs. Complex Audio

- The algorithm performs well for simple, structured beats, such as metronome sounds or drum tracks.
- Accuracy is high when detecting beats in audio with a single dominant tempo and minimal background noise.
- Performance deteriorates when applied to songs with complex compositions, multiple tempo changes, or significant vocal elements.

9.2 Challenges with Voice and Instrumental Complexity

- Songs with heavy vocals and varying bass/treble interfere with beat detection.
- Mixed frequencies from instruments and voice components make it difficult to isolate rhythmic patterns.
- The current filtering method does not sufficiently separate beats from other musical elements.

9.3 BPM Estimation Variability

- For steady, simple beats, BPM estimation is consistent and accurate.
- In songs with tempo variations, BPM fluctuates significantly, leading to incorrect estimations.

10 Challenges and Possible Improvements

10.1 Challenges Faced

10.1.1 Noise Interference

- Background noise or non-rhythmic segments may lead to incorrect beat detection.
- Erroneous frequencies must be eliminated.

10.1.2 Variability in Music Tempo

- Struggling to retain accuracies in the presence of complex rhythms or tempo-changing tracks.
- There should be a dynamic tempo-tracking algorithm.

10.1.3 Threshold Selection for Beat Detection

- It may not be possible for fixed thresholds on spectral flux or energy to suit all songs.
- Adaptive thresholding methods must be employed.

10.2 Possible Improvements

10.2.1 Adaptive Thresholding

- Apply a dynamic threshold on average energies to enhance beat detection across genres.

10.2.2 Machine Learning Integration

- Train an AI model to better detect beats by learning from a diverse set of music data.

10.2.3 Filter Design for Noise Reduction

- Employ bandpass filters to extract rhythmic content and reject unwanted noise.

11 Conclusion

In this project, we have employed a basic BPM detection system using DSP methods like FFT (Fast Fourier Transform), STE (Short-Time Energy), Spectral Flux (SF), and BPM (Beats Per Minute) estimation. The system can detect beats and estimate BPM for basic rhythmic patterns. This is an extremely basic implementation and is not likely to give a correct answer for complicated songs with changing tempos, background noise, or intricate beat patterns. The Current approach is best for steady beats but doesn't do well with dynamic or polyphonic music. Further enhancements like adaptive thresholding, incorporation of machine learning, and sophisticated filtering algorithms can enhance accuracy and make the system more robust for varying audio inputs.

References

[1] S. K. Mitra, *Digital Signal Processing: A Computer-Based Approach*, 4th ed. New York, NY, USA: McGraw-Hill, 2011.

[2] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.

[3] YouTube, "60 BPM Metronome," YouTube Video. Available: `https://youtu.be/gsJEMH_emBM`

12 MATLAB Code for Beat Detection

The following MATLAB code implements beat detection using audio processing techniques.

Listing 1: Beat Detection in MATLAB

```
% Read the audio file
[audio, fs] = audioread('Kick␣Drum␣Metronomes␣-␣60␣BPM.mp3');
audio = mean(audio, 2);

% Downsample to reduce computational load
targetFs = 2000;
if fs > targetFs
    audio = resample(audio, targetFs, fs);
    fs = targetFs;
end

% Applying a bandpass filter for beat-related frequencies -(20150 Hz)
bpFilt = designfilt('bandpassiir', 'FilterOrder', 6, ...
    'HalfPowerFrequency1', 20, 'HalfPowerFrequency2', 150, 'SampleRate', fs
        );
filteredAudio = filtfilt(bpFilt, audio);

% Parameters for FFT
```

```
frameSize = 1024;
hopSize = 512;
numFrames = floor((length(filteredAudio) - frameSize) / hopSize) + 1;
freqAxis = (0:(frameSize/2-1)) * (fs / frameSize);

% Initialize beat energy tracking
beatEnergy = zeros(numFrames, 1);

for frame = 1:numFrames

    startIdx = (frame - 1) * hopSize + 1;
    endIdx = startIdx + frameSize - 1;
    if endIdx > length(filteredAudio)
        break;
    end
    signalFrame = filteredAudio(startIdx:endIdx) .* hamming(frameSize);

    % Perform FFT
    fftResult = fft(signalFrame);
    fftMagnitude = abs(fftResult(1:frameSize/2));

    % Sum energy in the beat frequency range -(20150 Hz)
    freqRange = (freqAxis >= 20) & (freqAxis <= 150);
    beatEnergy(frame) = sum(fftMagnitude(freqRange).^2);
end

% Smooth the beat energy signal
smoothedBeatEnergy = movmean(beatEnergy, 5);

% Detect peaks in energy
threshold = 0.3 * max(smoothedBeatEnergy);
[~, beatLocs] = findpeaks(smoothedBeatEnergy, 'MinPeakHeight', threshold,
    ...
    'MinPeakDistance', round(fs * 60 / 120 / hopSize));

% Calculate BPM and number of beats
totalTime = length(filteredAudio) / fs;
beatCount = length(beatLocs);
bpm = (beatCount / totalTime) * 60;

% Display results
disp(['Number␣of␣beats␣detected:␣', num2str(beatCount)]);
disp(['Estimated␣BPM:␣', num2str(bpm)]);
```